Viridis Research — certification record

Run `runs/2026-10-28_P1` 2026-10-28 project P1, task P1-T001 engine `9ecce251ddf1`
GitHub run: `https://github.com/jdhart81/viridis-engine/actions/runs/40701505738`

The 8-connected bit-quad Euler formula for finite rasters: a Lean proof by vertex double counting
Justin Hart, Viridis LLC

Labels awarded by the verifier

Claim	Label	Claim text (as submitted)
C1	**PROVEN**	formal [P1.BitQuad]: For every finite raster B (a Finset of Z x Z), 4 * euler B = Q1 B - Q3 B - 2 * QD B, where euler B = |vertices| - (|hEdges| + |vEdges|) + |B| for the cubical complex K(B), and Q1, Q3, QD count the vertex-centred 2x2 windows with one pixel, three pixels, and a diagonal pair.
C2	**ASSUMED**	empirical: A brute-force recomputation of both sides of the bit-quad identity over all 65,536 subsets of a 4x4 box finds no counterexample.

Sealed statement P1.BitQuad

File `lean/ViridisStatements/P1/BitQuad.lean`, sha256 `cd3cdeba48f67f9545fb6945c87037893aa98b018922d3d0b7220cc389c98e23`, sealed by decision D-001. Axioms used by the proof: Classical.choice, Quot.sound, propext. Candidate sha256 `328a9a292c6c09accf3439cb81f2fce20d97efa7ee1eeb8a9acc9d72e6d3c77f`. Listing shown with symbol substitution; the exact bytes are in the repository.

```
import Mathlib

/-!
# P1 · L1: the bit-quad formula for an 8-connected raster

A raster is a finite set `B` of pixels in `ℤ × ℤ`. Pixel `(i, j)` stands for the
closed unit square `[i, i+1] × [j, j+1]`. `K B` is the union of the closed
squares of `B`. It is a cubical complex, and its vertices, edges and faces are
counted below. Two pixels that share only a corner meet in `K B`, which is the
8-connected reading of the foreground.

`Statement` says that for every finite raster,
`4 · χ(K B) = Q₁ − Q₃ − 2·Q_D`. Here `Q₁`, `Q₃` and `Q_D` count the 2×2 windows
that hold exactly one pixel of `B`, exactly three pixels of `B`, and exactly two
diagonally opposite pixels of `B` (Gray 1971).

This is the first rung (L1) of the P1 lemma ladder. See
`projects/P1/context/STATEMENT_PROPOSAL.md`.
-/

namespace ViridisStatements.P1.BitQuad

/-- A pixel, named by its lower-left lattice corner. -/
abbrev Pixel := ℤ × ℤ

/-- The four lattice corners of pixel `p`. -/
def corners (p : Pixel) : Finset (ℤ × ℤ) :=
  {(p.1, p.2), (p.1 + 1, p.2), (p.1, p.2 + 1), (p.1 + 1, p.2 + 1)}

/-- Vertices of `K B`: every corner of every pixel of `B`. -/
def vertices (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion corners

/-- Horizontal unit edges `[(i, j), (i + 1, j)]` of `K B`, named by their left
end. These are the bottom edge `(i, j)` and the top edge `(i, j + 1)` of each
pixel `(i, j)` of `B`. -/
def hEdges (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion fun p => {(p.1, p.2), (p.1, p.2 + 1)}

/-- Vertical unit edges `[(i, j), (i, j + 1)]` of `K B`, named by their lower
end. These are the left edge `(i, j)` and the right edge `(i + 1, j)` of each
pixel `(i, j)` of `B`. -/
def vEdges (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion fun p => {(p.1, p.2), (p.1 + 1, p.2)}

/-- The Euler characteristic `V − E + F` of `K B`. -/
def euler (B : Finset Pixel) : ℤ :=
  ((vertices B).card : ℤ) - (((hEdges B).card : ℤ) + (vEdges B).card) + B.card
```

```
/-- The 2×2 window centred on the lattice point `v`, as four membership bits in
this order: lower-left, lower-right, upper-left, upper-right. -/
def bits (B : Finset Pixel) (v : ℤ × ℤ) : Bool × Bool × Bool × Bool :=
  (decide ((v.1 - 1, v.2 - 1) ∈ B), decide ((v.1, v.2 - 1) ∈ B),
   decide ((v.1 - 1, v.2) ∈ B), decide ((v.1, v.2) ∈ B))

/-- The number of pixels of `B` in the window centred on `v`. -/
def windowCount (B : Finset Pixel) (v : ℤ × ℤ) : ℕ :=
  match bits B v with
  | (a, b, c, d) => a.toNat + b.toNat + c.toNat + d.toNat

/-- True when the window centred on `v` holds exactly two diagonally opposite
pixels of `B`. -/
def isDiagonal (B : Finset Pixel) (v : ℤ × ℤ) : Bool :=
  match bits B v with
  | (a, b, c, d) => (a && d && !b && !c) || (b && c && !a && !d)

/-- Windows holding exactly one pixel of `B`. Every window that meets `B` is
centred on a vertex of `K B`, so counting over `vertices B` counts each such
window exactly once. -/
def Q1 (B : Finset Pixel) : ℕ :=
  ((vertices B).filter fun v => windowCount B v = 1).card

/-- Windows holding exactly three pixels of `B`. -/
def Q3 (B : Finset Pixel) : ℕ :=
  ((vertices B).filter fun v => windowCount B v = 3).card

/-- Windows holding exactly two diagonally opposite pixels of `B`. -/
def QD (B : Finset Pixel) : ℕ :=
  ((vertices B).filter fun v => isDiagonal B v = true).card

/-- **Sealed target `P1.BitQuad`.** For every finite raster `B`,
`4 · χ(K B) = Q₁ − Q₃ − 2·Q_D`. -/
def Statement : Prop :=
  ∀ B : Finset Pixel, 4 * euler B = (Q1 B : ℤ) - Q3 B - 2 * QD B

/-! ## Sanity checks

These checks are proved by computation inside the sealed module, so the
definitions above mean what their comments say. -/

/-- A single pixel: `V = 4`, `E = 4`, `F = 1`, so `χ = 1`, and its four windows
each hold one pixel. -/
example : euler {(0, 0)} = 1 ∧ Q1 {(0, 0)} = 4 ∧ Q3 {(0, 0)} = 0 ∧ QD {(0, 0)} = 0 := by
  decide

/-- Two pixels that touch only at a corner form one 8-connected piece: `χ = 1`,
and the shared corner is a diagonal window. -/
example : euler {(0, 0), (1, 1)} = 1 ∧ QD {(0, 0), (1, 1)} = 1 ∧ Q1 {(0, 0), (1, 1)} = 6 := by
  decide

/-- A 2×2 solid block is contractible: `χ = 1`. -/
example : euler {(0, 0), (1, 0), (0, 1), (1, 1)} = 1 := by
  decide

/-- The 3×3 ring (eight pixels around one hole) has `χ = 0`. -/
example : euler {(0, 0), (1, 0), (2, 0), (0, 1), (2, 1), (0, 2), (1, 2), (2, 2)} = 0 := by
  decide

end ViridisStatements.P1.BitQuad
```

Other checks

- Lean toolchain leanprover/lean4:v4.28.0, Mathlib 8f9d9cff6bd7; kernel replay by `leanchecker`.
- References: 5 entries, 5 DOIs, 5 resolved, 0 unresolved, 0 unchecked.
- Agent paper: compiled.
- Statement proposals: 0.

How to read this record

ASSUMED: not verified (the default). **PROVEN**: a Lean 4 proof of a statement sealed before the attempt, compiled with the pinned toolchain, matched definitionally to the sealed statement, using only the axioms propext, Classical.choice and Quot.sound, and replayed through the kernel. **TESTED/REPLICATED**: reproduced by the

verifier (not yet enabled). **ENDORSED/ADOPTED**: granted by people. Novelty is never a label. **TRIVIAL** marks a sealed statement that one of the engine's probe tactics already closes.

This paper was drafted by an AI research agent (Claude, by Anthropic) running the Viridis research engine; formal proof search may use Aristotle (Harmonic). Justin Hart directs the research program. The labels on this page were assigned by an automated verifier, not by the agent, and only they carry certification. Claims in the paper that are not listed here as PROVEN are ASSUMED.

Reproduce everything from `REPRODUCE.md` in the run folder; the run's files are listed with hashes in `MANIFEST.sha256`.

The 8-connected bit-quad Euler formula for finite rasters: a Lean proof by vertex double counting

Justin Hart
Viridis LLC

28 October 2026

Abstract

For a finite set $B \subset \mathbb{Z}^2$ of pixels, let $K(B)$ be the union of the closed unit squares of B, regarded as a cubical complex with Euler characteristic $\chi(K(B)) = V - E + F$. Gray's bit-quad formula expresses this topological invariant through purely local counts: $4\chi(K(B)) = Q_1 - Q_3 - 2Q_D$, where Q_1, Q_3 and Q_D count the 2×2 windows holding exactly one pixel, exactly three pixels, and exactly two diagonally opposite pixels of B. We give a Lean 4 proof, against Mathlib, of the sealed statement `P1.BitQuad`, which states this identity for every finite raster. The proof is a double count over the vertices of $K(B)$. Each of F, E and V is written as a sum over vertices of indicator functions of translated sets. A single shift-counting lemma evaluates all eight such sums, and a pointwise identity is closed by a case split on the 16 window patterns, of which only the 15 non-empty ones can occur at a vertex. This is the first rung (L1) of a lemma ladder whose goal is to replace the topological hypothesis assumed in the machine-checked core of *Trees Cannot Cut* with a theorem about rasters.

1 Introduction

Hart's *Trees Cannot Cut* [3] argues that fuel breaks and habitat corridors split a forest landscape into separate compartments only through their cycles: an acyclic corridor leaves the fuel matrix connected. The graph theory behind this claim is machine-checked, but the step from topology to graph theory is carried as an explicit hypothesis (`FaceCount`), which says that the number of compartments equals the circuit rank plus one. The manuscript justifies it by Alexander duality. Mathlib [5] has neither Alexander duality nor a planar Euler formula in a form that applies here.

Fire and fuel-break data usually come as rasters, so project P1 aims to discharge the hypothesis in the raster setting. For a finite barrier raster B with 8-connected barriers and a 4-connected complement, the target identity is

$$\#\{\text{enclosed 4-components of } \mathbb{Z}^2 \setminus B\} \;=\; C_8(B) - \chi(K(B)).$$

The proposed route is a ladder of three lemmas. L1 is a local formula for $\chi(K(B))$. L2 says that adding one pixel preserves enclosed $- C_8 + \chi$. L3 is the bridge itself, by induction on $|B|$. This paper does L1: it gives a Lean proof of Gray's bit-quad formula in its 8-connected form. L1 matters for the rest of the ladder because it makes the change in χ under a single-pixel insertion a purely local quantity: only the four windows around the new pixel change.

2 Background and related work

Gray [2] introduced locally countable properties of binary images, among them area, perimeter and the Euler number. He showed that the Euler number can be computed from counts of 2×2 "bit-quad" patterns, with different weightings for 4- and 8-connectivity of the foreground. The (8,4) pairing of connectivity for foreground and background, and the role it plays in digital Jordan-type theorems, are surveyed by Kong and Rosenfeld [4]. In the 8-connected reading, two pixels that share only a corner touch in $K(B)$. The two diagonal patterns then contribute -2 to 4χ, because their shared corner is one vertex where four edges meet. Our Lean development uses Lean 4 [1] and Mathlib [5]. We are not aware of an earlier machine-checked proof of the bit-quad formula, but we have not searched exhaustively and make no claim of novelty.

3 Setting: the sealed statement

The statement module `ViridisStatements.P1.BitQuad` fixes the following definitions. A pixel $p = (i, j) \in \mathbb{Z}^2$ stands for the closed square $[i, i+1] \times [j, j+1]$.

- $\mathrm{vertices}(B) = \bigcup_{p \in B} \mathrm{corners}(p)$, where $\mathrm{corners}(i, j) = \{(i, j), (i+1, j), (i, j+1), (i+1, j+1)\}$.
- $\mathrm{hEdges}(B) = \bigcup_{(i,j) \in B} \{(i, j), (i, j+1)\}$ (horizontal edges named by their left end) and $\mathrm{vEdges}(B) = \bigcup_{(i,j) \in B} \{(i, j), (i+1, j)\}$ (vertical edges named by their lower end).
- The Euler characteristic, computed in $\mathbb{Z}$:

$$\chi(B) = |\mathrm{vertices}(B)| - (|\mathrm{hEdges}(B)| + |\mathrm{vEdges}(B)|) + |B|.$$

- The window at a lattice point $v = (x, y)$ consists of four bits, $a = [(x-1, y-1) \in B]$, $b = [(x, y-1) \in B]$, $c = [(x-1, y) \in B]$ and $d = [(x, y) \in B]$, with $n(v) = a + b + c + d$.
- Q_1, Q_3 and Q_D count the vertices v with $n(v) = 1$, with $n(v) = 3$, and with $\{a, d\}$ or $\{b, c\}$ as the only set bits.

Every window that meets B is centred on a vertex of $K(B)$, so counting over the vertices counts every such window exactly once.

Theorem 1 (`P1.BitQuad`)**.** *For every finite $B \subset \mathbb{Z}^2$, $4\,\chi(K(B)) = Q_1(B) - Q_3(B) - 2\,Q_D(B)$.*

4 Proof architecture

Write $\mathcal{V} = \mathrm{vertices}(B)$, and use $[P] \in \{0, 1\}$ for the indicator of a proposition P.

Lemma 2 (Edge membership)**.** $(i, j) \in \mathrm{hEdges}(B) \iff (i, j) \in B \vee (i, j-1) \in B$*, and* $(i, j) \in \mathrm{vEdges}(B) \iff (i, j) \in B \vee (i-1, j) \in B$*.*

Lemma 3 (Shift counting)**.** *Let $S, \mathcal{V} \subset \mathbb{Z}^2$ be finite and $s \in \mathbb{Z}^2$. If $x + s \in \mathcal{V}$ for every $x \in S$, then $\sum_{v \in \mathcal{V}} [v - s \in S] = |S|$.*

Proof. The map $v \mapsto v - s$ is a bijection from $\{v \in \mathcal{V} : v - s \in S\}$ onto S. Its inverse is $x \mapsto x + s$, which lands in $\mathcal{V}$ by hypothesis. □

Lemma 3 does all of the double counting. Every pixel $x \in B$ has its four corners x, $x+(1,0)$, $x+(0,1)$ and $x+(1,1)$ in $\mathcal{V}$. Applying the lemma with the four shifts gives

$$4F = \sum_{v \in \mathcal{V}} n(v).$$

By Lemma 2, both ends x and $x+(1,0)$ of a horizontal edge are corners of the pixel above or below it, and likewise for vertical edges. Write the arms of v as left $[v-(1,0) \in \mathrm{hEdges}]$, right $[v \in \mathrm{hEdges}]$, down $[v-(0,1) \in \mathrm{vEdges}]$ and up $[v \in \mathrm{vEdges}]$. Applying the lemma once more for each arm gives

$$2E = \sum_{v \in \mathcal{V}} \mathrm{arms}(v), \qquad V = \sum_{v \in \mathcal{V}} 1.$$

Hence $4\chi = \sum_{v \in \mathcal{V}} \big(4 - 2\,\mathrm{arms}(v) + n(v)\big)$. Lemma 2 expresses each arm through the window bits. The left arm is $a \vee c$, the right arm $b \vee d$, the down arm $a \vee b$ and the up arm $c \vee d$. The summand therefore depends only on (a,b,c,d), and Table 1 evaluates it. Every vertex is a corner of some pixel, which must be one of the four window cells, so the empty pattern never occurs. On the other 15 patterns the summand equals $[n=1] - [n=3] - 2[\text{diagonal}]$. Summing over $\mathcal{V}$ proves Theorem 1.

Pattern class	#patterns	n	arms	$4 - 2\,\mathrm{arms} + n$
empty (cannot occur)	1	0	0	4
one pixel	4	1	2	1
adjacent pair	4	2	3	0
diagonal pair	2	2	4	-2
three pixels	4	3	4	-1
full	1	4	4	0

Table 1: The local summand on the 16 window patterns.

5 The Lean development

The candidate file `proofs/P1.BitQuad.lean` (145 lines) follows the architecture above.

- `mem_vertices`, `mem_corners`, `mem_hEdges` and `mem_vEdges` unfold the membership predicates (Lemma 2).
- `card_shift` proves Lemma 3 with `Finset.card_bij`. Its corollary `sum_shift` restates it as a sum of $\{0,1\}$-valued integer indicators via `Finset.sum_boole`.
- In `main`, the eight instances of `sum_shift` (four for faces, two for each edge direction) and the three expressions of Q_1, Q_3 and Q_D as indicator sums reduce the goal to a single `Finset.sum_congr`. The pointwise goal is closed by `by_cases` on the four window memberships followed by `simp`, which also uses the fact that the window at a vertex is non-empty.

The brief suggested evaluating the 16 patterns with `decide`. That proved unnecessary: once the window memberships are case-split, `simp` closes all 16 branches directly. The file uses no `decide` on large objects and no `native_decide`. The project checker (`viridis-lean-check`) reports that the file compiles under Lean `v4.28.0` with the pinned Mathlib, matches the sealed statement, depends

only on `propext`, `Classical.choice` and `Quot.sound`, and replays through the kernel. Whether the claim is accepted is decided by the project's independent verifier; see its certification page. The complete file follows.

```
import Mathlib
import ViridisStatements.P1.BitQuad

namespace Candidate

open ViridisStatements.P1.BitQuad

/-- Membership in `vertices`. -/
theorem mem_vertices (B : Finset Pixel) (v : ℤ × ℤ) :
    v ∈ vertices B ↔ ∃ p ∈ B, v ∈ corners p := by
  unfold vertices
  simp only [Finset.mem_biUnion]

theorem mem_corners (p v : ℤ × ℤ) :
    v ∈ corners p ↔ v = (p.1, p.2) ∨ v = (p.1 + 1, p.2) ∨ v = (p.1, p.2 + 1) ∨
      v = (p.1 + 1, p.2 + 1) := by
  unfold corners
  simp only [Finset.mem_insert, Finset.mem_singleton]

/-- A horizontal edge is present iff the pixel above or below it is in `B`. -/
theorem mem_hEdges (B : Finset Pixel) (h : ℤ × ℤ) :
    h ∈ hEdges B ↔ (h.1, h.2) ∈ B ∨ (h.1, h.2 - 1) ∈ B := by
  unfold hEdges
  simp only [Finset.mem_biUnion, Finset.mem_insert, Finset.mem_singleton]
  constructor
  · rintro ⟨p, hp, rfl | rfl⟩
    · left; simpa using hp
    · right; simpa using hp
  · rintro (h1 | h1)
    · exact ⟨_, h1, Or.inl rfl⟩
    · exact ⟨_, h1, Or.inr (by simp)⟩

/-- A vertical edge is present iff the pixel right or left of it is in `B`. -/
theorem mem_vEdges (B : Finset Pixel) (e : ℤ × ℤ) :
    e ∈ vEdges B ↔ (e.1, e.2) ∈ B ∨ (e.1 - 1, e.2) ∈ B := by
  unfold vEdges
  simp only [Finset.mem_biUnion, Finset.mem_insert, Finset.mem_singleton]
  constructor
  · rintro ⟨p, hp, rfl | rfl⟩
    · left; simpa using hp
    · right; simpa using hp
  · rintro (h1 | h1)
    · exact ⟨_, h1, Or.inl rfl⟩
    · exact ⟨_, h1, Or.inr (by simp)⟩

/-- Shift-counting: if every translate `x + (a, b)` of `S` lies in `V`, then
counting the `v ∈ V` with `v - (a, b) ∈ S` gives `|S|`. -/
theorem card_shift (V S : Finset (ℤ × ℤ)) (a b : ℤ)
    (hS : ∀ x ∈ S, (x.1 + a, x.2 + b) ∈ V) :
    (V.filter fun v => (v.1 - a, v.2 - b) ∈ S).card = S.card := by
  apply Finset.card_bij (fun v _ => (v.1 - a, v.2 - b))
  · intro v hv
    exact (Finset.mem_filter.mp hv).2
  · intro v₁ _ v₂ _ h
    simp only [Prod.mk.injEq] at h
```

```
    ext <;> omega
  · intro x hx
    refine ⟨(x.1 + a, x.2 + b), Finset.mem_filter.mpr ⟨hS x hx, ?_⟩, ?_⟩
    · simpa using hx
    · simp

theorem sum_shift (V S : Finset (ℤ × ℤ)) (a b : ℤ)
    (hS : ∀ x ∈ S, (x.1 + a, x.2 + b) ∈ V) :
    (∑ v ∈ V, if (v.1 - a, v.2 - b) ∈ S then (1 : ℤ) else 0) = S.card := by
  rw [Finset.sum_boole, card_shift V S a b hS]

theorem main : ViridisStatements.P1.BitQuad.Statement := by
  intro B
  set V := vertices B with hV
  -- inclusion facts
  have hB : ∀ x ∈ B, ∀ a b : ℤ, (a = 0 ∨ a = 1) → (b = 0 ∨ b = 1) →
      (x.1 + a, x.2 + b) ∈ V := by
    intro x hx a b ha hb
    rw [hV, mem_vertices]
    refine ⟨x, hx, ?_⟩
    rw [mem_corners]
    rcases ha with rfl | rfl <;> rcases hb with rfl | rfl <;> simp
  have hH : ∀ x ∈ hEdges B, ∀ a : ℤ, (a = 0 ∨ a = 1) → (x.1 + a, x.2 + 0) ∈ V := by
    intro x hx a ha
    rw [mem_hEdges] at hx
    rcases hx with hx | hx
    · have := hB _ hx a 0 ha (Or.inl rfl)
      simpa using this
    · have := hB _ hx a 1 ha (Or.inr rfl)
      simpa using this
  have hVe : ∀ x ∈ vEdges B, ∀ b : ℤ, (b = 0 ∨ b = 1) → (x.1 + 0, x.2 + b) ∈ V := by
    intro x hx b hb
    rw [mem_vEdges] at hx
    rcases hx with hx | hx
    · have := hB _ hx 0 b (Or.inl rfl) hb
      simpa using this
    · have := hB _ hx 1 b (Or.inr rfl) hb
      simpa using this
  -- the four expressions as sums over V
  have eV : ((V.card : ℤ)) = ∑ v ∈ V, (1 : ℤ) := by simp
  have eF11 := sum_shift V B 1 1 (fun x hx => hB x hx 1 1 (Or.inr rfl) (Or.inr rfl))
  have eF01 := sum_shift V B 0 1 (fun x hx => hB x hx 0 1 (Or.inl rfl) (Or.inr rfl))
  have eF10 := sum_shift V B 1 0 (fun x hx => hB x hx 1 0 (Or.inr rfl) (Or.inl rfl))
  have eF00 := sum_shift V B 0 0 (fun x hx => hB x hx 0 0 (Or.inl rfl) (Or.inl rfl))
  have eH10 := sum_shift V (hEdges B) 1 0 (fun x hx => hH x hx 1 (Or.inr rfl))
  have eH00 := sum_shift V (hEdges B) 0 0 (fun x hx => hH x hx 0 (Or.inl rfl))
  have eV01 := sum_shift V (vEdges B) 0 1 (fun x hx => hVe x hx 1 (Or.inr rfl))
  have eV00 := sum_shift V (vEdges B) 0 0 (fun x hx => hVe x hx 0 (Or.inl rfl))
  have eQ1 : ((Q1 B : ℕ) : ℤ) = ∑ v ∈ V, if windowCount B v = 1 then (1 : ℤ) else 0 := by
    rw [Finset.sum_boole]; rfl
  have eQ3 : ((Q3 B : ℕ) : ℤ) = ∑ v ∈ V, if windowCount B v = 3 then (1 : ℤ) else 0 := by
    rw [Finset.sum_boole]; rfl
  have eQD : ((QD B : ℕ) : ℤ) = ∑ v ∈ V, if isDiagonal B v = true then (1 : ℤ) else 0 := by
    rw [Finset.sum_boole]; rfl
  have key : 4 * euler B = ∑ v ∈ V,
      ((4 : ℤ) - 2 * ((if (v.1 - 1, v.2 - 0) ∈ hEdges B then 1 else 0)
        + (if (v.1 - 0, v.2 - 0) ∈ hEdges B then 1 else 0)
        + (if (v.1 - 0, v.2 - 1) ∈ vEdges B then 1 else 0)
        + (if (v.1 - 0, v.2 - 0) ∈ vEdges B then 1 else 0))
```

```
        + ((if (v.1 - 1, v.2 - 1) ∈ B then 1 else 0)
        + (if (v.1 - 0, v.2 - 1) ∈ B then 1 else 0)
        + (if (v.1 - 1, v.2 - 0) ∈ B then 1 else 0)
        + (if (v.1 - 0, v.2 - 0) ∈ B then 1 else 0))) := by
    simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib, ← Finset.mul_sum]
    rw [eF11, eF01, eF10, eF00, eH10, eH00, eV01, eV00]
    unfold euler
    rw [← hV]
    simp
    ring
  rw [key, eQ1, eQ3, eQD, ← Finset.sum_sub_distrib, Finset.mul_sum,
    ← Finset.sum_sub_distrib]
  apply Finset.sum_congr rfl
  intro v hv
  obtain ⟨x, y⟩ := v
  have hne : (x - 1, y - 1) ∈ B ∨ (x, y - 1) ∈ B ∨ (x - 1, y) ∈ B ∨ (x, y) ∈ B := by
    rw [hV, mem_vertices] at hv
    obtain ⟨p, hp, hc⟩ := hv
    rw [mem_corners] at hc
    rcases hc with h | h | h | h <;> simp only [Prod.mk.injEq] at h <;> obtain ⟨rfl, rfl⟩ := h
    · right; right; right; exact hp
    · right; right; left; simpa using hp
    · right; left; simpa using hp
    · left; simpa using hp
  unfold windowCount isDiagonal bits
  simp only [sub_zero, mem_hEdges, mem_vEdges]
  by_cases ha : (x - 1, y - 1) ∈ B <;> by_cases hb : (x, y - 1) ∈ B <;>
    by_cases hc : (x - 1, y) ∈ B <;> by_cases hd : (x, y) ∈ B <;>
    simp [ha, hb, hc, hd] at hne ⊢

end Candidate
```

6 Results

- We give a Lean proof of Theorem 1, the sealed statement `P1.BitQuad`, for all finite rasters.
- As an independent computational check, which is evidence and not a proof, the script `code/bitquad_check.py` recomputes both sides for all $2^{16} = 65{,}536$ subsets of a 4×4 box. It finds no counterexample (`results/bitquad_4x4.json`). Euler characteristics from -1 to 4 occur in that box.

7 How L1 serves the L2/L3 bridge

Let $p \notin B$ and $B' = B \cup \{p\}$. The windows whose bits differ between B and B' are exactly the four windows centred on the corners of p. By Theorem 1 applied to B and to B', $4(\chi(B') - \chi(B))$ equals the change in $Q_1 - Q_3 - 2Q_D$ over those four windows. That change depends only on the 3×3 neighbourhood of p. This is the "local half" of L2. The non-local half, the change in C_8 and in the number of enclosed 4-components of the complement, still needs a separate argument: a pixel joining two 8-components can close off a new bounded region. L1 contributes nothing there. The negative controls listed in the project context (the solid 2×2 block, whose 8-neighbour graph K_4 has circuit rank 3 yet encloses nothing, and the four-pixel N/S/E/W cross enclosing its centre) show that L2 must be stated in terms of $\chi(K(B))$ and not in terms of the graph circuit rank.

8 Limitations

- The theorem concerns the combinatorial counts $V - E + F$ as defined in the sealed module. The development does not show that this number is the Euler characteristic of the topological space $K(B) \subset \mathbb{R}^2$. That identification is standard but is not formalised here, and L2/L3 are designed not to need it.
- Only the 8-connected weighting of the bit quads is treated. The 4-connected variant, with $+2Q_D$, is not formalised.
- L2 and L3 remain open, so the `FaceCount` hypothesis of *Trees Cannot Cut* is not yet discharged.

9 Reproducibility

The proof file is `proofs/P1.BitQuad.lean`. It imports only `Mathlib` and the sealed module `ViridisStatements.P1.BitQuad` (sha256 `cd3cdeba48f6...8e23`), and it builds with toolchain `leanprover/lean4:v4.28.0` and Mathlib commit `8f9d9cff6bd728b17a24e163c9402775d9e6a365`. The check takes about 20 seconds. The brute-force script runs with the Python 3 standard library: `python3 code/bitquad_check.py results/bitquad_4x4.json`. No external data are used.

Changelog

- 2026-10-28 (P1-T001): first version of the paper. Lean proof of L1 (`P1.BitQuad`), proof architecture, brute-force check on the 4×4 box, and discussion of the role of L1 in L2/L3.

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[2] Stephen B. Gray. Local properties of binary images in two dimensions. *IEEE Transactions on Computers*, C-20(5):551–561, 1971. doi: 10.1109/T-C.1971.223289.

[3] Justin D. Hart. Trees cannot cut: Circuit rank as the design variable reconciling habitat connectivity and fuel discontinuity in managed forest landscapes. Zenodo, Viridis LLC, 2026.

[4] T. Yung Kong and Azriel Rosenfeld. Digital topology: Introduction and survey. *Computer Vision, Graphics, and Image Processing*, 48(3):357–393, 1989. doi: 10.1016/0734-189X(89)90147-3.

[5] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.